

Introduction

Q-Learning is a model-free, off-policy reinforcement learning algorithm that learns the optimal action-value function $Q(s, a)$. It updates Q-values using the Bellman optimality principle by considering the maximum future reward of the next state. The agent selects actions using an exploration–exploitation strategy such as ϵ -greedy. Over time, the learned Q-table converges to the optimal policy for discrete environments.

2. Q-Function

The Q-function is defined as:

$$Q(s, a) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right]$$

Where:

- s = state
- a = action
- r_t = reward at time t
- γ = discount factor ($0 \leq \gamma \leq 1$)

It represents the **quality (Q-value)** of performing action a in state s .

3. Learning Objective

The goal of Q-Learning is to learn the **optimal Q-function** $Q^*(s, a)$, which satisfies the **Bellman Optimality Equation**:

$$Q^*(s, a) = r + \gamma \max_{a'} Q^*(s', a')$$

Where:

- s' = next state
- a' = possible next action

4. Q-Learning Update Rule

At each step, the Q-value is updated using:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Where:

- α = learning rate
- r = reward received
- γ = discount factor
- $\max_{a'} Q(s', a')$ = estimated best future reward

Advantages

- Simple to understand and implement
 - Guaranteed to converge (under proper conditions)
 - Does not require a model of the environment
 - Works well for **discrete state and action spaces**
-

Disadvantage

- Poor scalability for large state spaces (Q-table becomes huge)
- Slow learning in complex environments
- Requires careful tuning of ϵ , α , and γ
- Not ideal for continuous action spaces

Q-Learning Algorithm - Reinforcement learning

Q learning algorithm

For each s, a initialize the table entry $\hat{Q}(s, a)$ to zero.

Observe the current state s

Do forever:

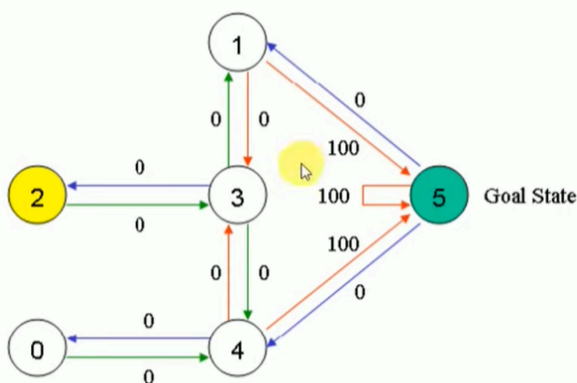
- Select an action a and execute it
- Receive immediate reward r
- Observe the new state s'
- Update the table entry for $\hat{Q}(s, a)$ as follows:

$$\hat{Q}(s, a) \leftarrow r + \gamma \max_{a'} \hat{Q}(s', a')$$

- $s \leftarrow s'$

Q-Learning - Reinforcement Learning - Solved Example - 1

- We can put the state diagram and the instant reward values into the following reward table, "matrix R". The -1's in the table represent null values (i.e.; where there isn't a link between nodes). For example, State 0 cannot go to State 1.



State	Action					
	0	1	2	3	4	5
0	-1	-1	-1	-1	0	-1
1	-1	-1	-1	0	-1	100
2	-1	-1	-1	0	-1	-1
3	-1	0	0	-1	0	-1
4	0	-1	-1	0	-1	100
5	-1	0	-1	-1	0	100

Q-Learning - Reinforcement Learning - Solved Example - 1

- Now let's imagine what would happen if our agent were in state 5 (next state).
- Look at the sixth row of the reward matrix R (i.e. state 5).
- It has 3 possible actions: go to state 1, 4 or 5.
- $Q(\text{state}, \text{action}) = R(\text{state}, \text{action}) + \text{Gamma} * \text{Max}[Q(\text{next state}, \text{all actions})]$

$$R = \begin{array}{c|cccccc} & \text{Action} & & & & & \\ \hline \text{State} & 0 & 1 & 2 & 3 & 4 & 5 \\ \hline 0 & -1 & -1 & -1 & -1 & 0 & -1 \\ 1 & -1 & -1 & -1 & 0 & -1 & 100 \\ 2 & -1 & -1 & -1 & 0 & -1 & -1 \\ 3 & -1 & 0 & 0 & -1 & 0 & -1 \\ 4 & 0 & -1 & -1 & 0 & -1 & 100 \\ 5 & -1 & 0 & -1 & -1 & 0 & 100 \end{array}$$

$$Q = \begin{array}{c|cccccc} & 0 & 1 & 2 & 3 & 4 & 5 \\ \hline 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 & 0 & 0 & 0 \\ 4 & 0 & 0 & 0 & 0 & 0 & 0 \\ 5 & 0 & 0 & 0 & 0 & 0 & 0 \end{array}$$

Now let's imagine what would happen if our agent were in state 5 (next state).

Look at the sixth row of the reward matrix R (i.e. state 5).

It has 3 possible actions: go to state 1, 4 or 5.

$$Q(\text{state}, \text{action}) = R(\text{state}, \text{action}) + \text{Gamma} * \text{Max}[Q(\text{next state}, \text{all actions})]$$

$$Q(1, 5) = R(1, 5) + 0.8 * \text{Max}[Q(5, 1), Q(5, 4), Q(5, 5)] = 100 + 0.8 * 0 = 100$$

Q-Learning - Reinforcement Learning - Solved Example - 1

- For the next episode, we randomly choose the initial state – say 3 (can go to 1, 2 & 4)

$$R = \begin{array}{c|cccccc} & \text{Action} & & & & & \\ \hline \text{State} & 0 & 1 & 2 & 3 & 4 & 5 \\ \hline 0 & -1 & -1 & -1 & -1 & 0 & -1 \\ 1 & -1 & -1 & -1 & 0 & -1 & 100 \\ 2 & -1 & -1 & -1 & 0 & -1 & -1 \\ 3 & -1 & 0 & 0 & -1 & 0 & -1 \\ 4 & 0 & -1 & -1 & 0 & -1 & 100 \\ 5 & -1 & 0 & -1 & -1 & 0 & 100 \end{array}$$

Q-Learning - Reinforcement Learning - Solved Example -

- Now we imagine that we are in state 1 (next state).
- Look at the second row of reward matrix R (i.e. state 1).
- It has 2 possible actions: go to state 3 or state 5.
- Then, we compute the Q value:
- $Q(\text{state}, \text{action}) = R(\text{state}, \text{action}) + \text{Gamma} * \text{Max}[Q(\text{next state}, \text{all actions})]$
- $Q(3, 1) = R(3, 1) + 0.8 * \text{Max}[Q(1, 3), Q(1, 5)] = 0 + 0.8 * \text{Max}(0, 100) = 80$

		Action					
State		0	1	2	3	4	5
0	$R =$	-1	-1	-1	-1	0	-1
1		-1	-1	-1	0	-1	100
2		-1	-1	-1	0	-1	-1
3		-1	0	0	-1	0	-1
4		0	-1	-1	0	-1	100
5		-1	0	-1	-1	0	100

		0	1	2	3	4	5
$Q =$		0	0	0	0	0	0
0		0	0	0	0	0	0
1		0	0	0	0	0	100
2		0	0	0	0	0	0
3		0	0	0	0	0	0
4		0	0	0	0	0	0
5		0	0	0	0	0	0

Q-Learning - Reinforcement Learning - Solved Example

- If our agent learns more through further episodes, it will finally reach convergence values in matrix Q like:

		0	1	2	3	4	5
$Q =$		0	0	0	0	80	0
0		0	0	0	64	0	100
1		0	0	0	64	0	0
2		0	80	51	0	80	0
3		64	0	0	64	0	100
4		0	80	0	0	80	100
5		0	80	0	0	80	100

- Tracing the best sequences of states is as simple as following the links with the highest values at each state.

